# مدل‌سازی شناختی

جزوه‌ی کامل درس

هفته‌ی ۳: برازش مدل با کمترین مربعات: سطح خطا و بهینه‌سازی

# فهرست مطالب


**۱ روش‌های برآورد پارامتر** ۱

۱.۱ هفته‌ی ۳: برازش مدل با کمترین مربعات: سطح خطا و بهینه‌سازی ۱

۱.۱.۱ واژه‌های کلیدی ۱

۱.۱.۲ اهداف یادگیری ۲

۱.۱.۳ مقدمه ۳

۱.۱.۴ جزوه‌ی درس ۳

۱.۱.۵ مثال حل‌شده ۹

۱.۱.۶ lab (۲ ساعت) ۱۰

۱.۱.۷ ارتباط با هوش مصنوعی و LLMها ۱۵

۱.۱.۸ اشتباهات رایج و برداشت‌های نادرست ۱۵

۱.۱.۹ خودآزمایی ۱۶

۱.۱.۱۰ تکلیف ۱۷

۱.۱.۱۱ مطالعه‌ی بیشتر ۱۷

# فصل ۱

# روش‌های برآورد پارامتر

## ۱.۱ هفته‌ی ۳: برازش مدل با کمترین مربعات: سطح خطا و بهینه‌سازی

**فصل ۲: روش‌های برآورد پارامتر** · درس ۱ ساعت · lab ۲ ساعت **داده‌ی اصلی این هفته:** آزمایش فراموشی شبیه‌سازی‌شده از هفته‌ی ۲ (مدل توانی، $A^* = 0.9$، $d^* = 0.5$، $n = 50$ آیتم در ۸ فاصله). آن را درست برازش می‌کنیم و با برازش‌های خطی‌شده‌ی هفته‌ی قبل مقایسه می‌کنیم.

---

### ۱.۱.۱ واژه‌های کلیدی

واژه‌های مهم این درس با معادل فارسی‌شان (حتی اگر در درس‌های قبل آمده باشند).

| English | فارسی | توضیح کوتاه |
|---|---|---|
| fitting / fit | برازش | تنظیم پارامترها تا مدل با داده جور شود |
| least squares | کمترین مربعات | روش برازش با کمینه کردن مجموع مربع خطاها |
| error surface | سطح خطا | خطا به‌عنوان تابعی از پارامترها |
| optimizer | بهینه‌ساز | الگوریتمی که کمینه‌ی تابع را پیدا می‌کند |
| parameter | پارامتر | کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند |
| estimator / estimate | برآورد / برآوردگر | مقدار به‌دست‌آمده برای پارامتر / روش به دست آوردن آن |
| forgetting curve | منحنی فراموشی | کاهش یادآوری با گذشت زمان |
| power function / exponential | تابع نمایی / توانی | دو شکل رایج برای منحنی فراموشی و یادگیری |

| English | فارسی | توضیح کوتاه |
|---|---|---|
| RMSD / SSE | مجموع مربعات خطا / ریشه‌ی میانگین مربع انحراف | معیارهای فاصله‌ی داده و پیش‌بینی |
| noise | نویز | تغییرات تصادفی در شواهد یا در داده |
| parameter trade-off | بده‌بستان پارامترها | تغییر یک پارامتر که با پارامتر دیگر جبران می‌شود |
| grid approximation | تقریب شبکه‌ای (grid) | محاسبه‌ی پسین روی شبکه‌ای از مقادیر |
| Nelder–Mead | روش Nelder–Mead | بهینه‌ساز بدون نیاز به مشتق |
| Bayes factor (BF) | عامل بیز (Bayes factor) | نسبت درست‌نمایی‌های حاشیه‌ای دو مدل |
| local minimum | کمینه‌ی محلی | نقطه‌ای پایین‌تر از اطراف ولی نه بهترین نقطه |
| constraints / bounds | کران‌ها / قیدها | محدودیت‌های مجاز برای مقدار پارامترها |
| reparameterization | بازپارامتری‌سازی | نوشتن پارامتر در مقیاس دیگر، مثلاً $\lambda = e^{\eta}$ |
| multi-start | شروع چندگانه (multi-start) | اجرای بهینه‌ساز از چند نقطه‌ی شروع |
| maximum likelihood estimate (MLE) | برآورد بیشینه‌ی درست‌نمایی | پارامترهایی که داده را محتمل‌ترین می‌کنند |
| likelihood | درست‌نمایی | احتمال داده به‌عنوان تابعی از پارامترها |
| gradient | گرادیان | جهت و شیب تغییر تابع |
| convergence | همگرایی | رسیدن زنجیره‌ها به توزیع هدف |
| large language model (LLM) | مدل بزرگ زبانی | شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند |
| scaling law | قانون مقیاس | رابطه‌ی توانی بین اندازه‌ی مدل و خطا |

---

### ۲.۱.۱ اهداف یادگیری

در پایان این هفته، دانشجویان می‌توانند:

۱. SSE، MSE و RMSD را به‌عنوان توابع ناهمخوانی (discrepancy functions) **تعریف کنند** و **توضیح دهند** که کمینه کردن آن‌ها چه فرضی درباره‌ی نویز دارد.
۲. سطح خطای یک مدل دوپارامتری را **رسم کنند** و شکل آن (کمینه، دره، فلات) را بر حسب trade-off پارامترها **تفسیر کنند**.
۳. برازش کمترین مربعات را با جستجوی شبکه‌ای و با `scipy.optimize.minimize` (Nelder–Mead و L-BFGS-B) **پیاده‌سازی کنند** و نقاط قوت و حالت‌های شکست الگوریتم‌ها را **مقایسه کنند**.
۴. شکست‌های رایج برازش را **تشخیص دهند** و **رفع کنند**: مقدار شروع بد، فلات‌ها، کمینه‌های محلی، و پارامترهای خارج از دامنه (با کران، reparameterization و multi-start).
۵. یک برازش کمترین مربعات را با RMSD و باقی‌مانده‌ها **ارزیابی کنند** و پارامترهای برازش‌شده را **تفسیر کنند**.

### ۳.۱.۱ مقدمه

هفته‌ی قبل منحنی‌های فراموشی را با کشیدن یک خط راست از میان نسبت‌های لگاریتمی‌شده برازش کردیم. مدل توانی مدل واقعی بود، ولی برآوردهایش آشکارا خطا داشتند ($\hat{A} = 0.80$، $\hat{d} = 0.37$ به‌جای $0.9$ و $0.5$). این روش هر وقت یک شمارش صفر باشد از کار می‌افتد، و خطاها را روی مقیاس غلط (لگاریتمی) کمینه می‌کند.

راه‌حل از نظر مفهومی ساده است: **پارامترهایی را انتخاب کن که پیش‌بینی‌های مدل را، روی مقیاس خود داده، تا جای ممکن به داده نزدیک کنند.** «تا جای ممکن نزدیک» به یک عدد نیاز دارد: یک تابع ناهمخوانی. «پارامترها را انتخاب کن» به یک رویه‌ی جستجو نیاز دارد: یک الگوریتم بهینه‌سازی. این هفته هر دو را پوشش می‌دهد.

دلیل عمیق‌تری هم هست که این برای علوم شناختی مهم است. پارامترهای مدل اندازه‌گیری‌های ما از سازه‌های پنهان‌اند (نرخ زوال، احتیاط، حساسیت). رویه‌ی برازش ضعیف اندازه‌گیری‌های ضعیف می‌دهد، و هر نتیجه‌ای که از آن‌ها گرفته شود (تفاوت گروه‌ها، همبستگی با فعالیت مغز، مقایسه‌ی مدل‌ها) آن خطا را به ارث می‌برد. بدتر اینکه، همان‌طور که خواهیم دید، یک بهینه‌ساز می‌تواند «موفقیت» گزارش کند در حالی که جواب بی‌معنی برمی‌گرداند.

#### این موضوع کجا به کار می‌رود؟

- **هر مطالعه‌ی مبتنی بر مدل در روان‌شناسی** یک مرحله‌ی برازش دارد. منحنی‌های یادگیری، منحنی‌های فراموشی، تابع‌های روان‌سنجی :psychometric function) اینکه تشخیص با قوی‌تر شدن محرک چطور بهتر می‌شود) و منحنی‌های دوز–پاسخ در روان‌داروشناسی همه با کمینه کردن یک معیار خطا برازش می‌شوند.
- **پژوهش‌های بالینی و رشدی** پارامترهای برازش‌شده را بین گروه‌ها مقایسه می‌کنند (مثلاً نرخ فراموشی بیماران در برابر گروه کنترل). اگر برازش بد باشد یا روی یک کران گیر کرده باشد، مقایسه‌ی گروه‌ها غلط است.
- **علوم اعصاب** منحنی‌های تنظیم (tuning curve) نورون‌ها و تابع پاسخ همودینامیک در fMRI را با همین بهینه‌سازها برازش می‌کند.
- **یادگیری ماشین** مدل‌ها را با کمینه کردن یک تابع زیان (loss) با روش‌های گرادیانی آموزش می‌دهد؛ واژه‌ها (سطح زیان، نرخ یادگیری، کمینه‌ی محلی) همان واژه‌های این هفته‌اند.

---

### ۴.۱.۱ جزوه‌ی درس

**به زبان ساده** مدل چند پیچ تنظیم (پارامتر) دارد. برازش یعنی چرخاندن پیچ‌ها تا منحنی مدل تا جای ممکن از نزدیک نقطه‌های داده بگذرد. برای اینکه کامپیوتر این کار را بکند دو چیز لازم داریم. اول، یک عدد که بگوید برازش فعلی چقدر بد است: **مجموع مربعات خطا** (SSE)، یعنی جمع مربع فاصله‌ی عمودی نقطه‌های داده از منحنی. دوم، روشی برای چرخاندن پیچ‌ها که این عدد را کم کند: یک **بهینه‌ساز** (optimizer). SSE را مثل یک منظره روی همه‌ی تنظیم‌های ممکن پیچ‌ها تصور کنید، با تپه‌ها (برازش بد) و دره‌ها (برازش خوب)؛ بهینه‌ساز یک کوهنورد است که دنبال پایین‌ترین نقطه می‌گردد. مشکل وقتی پیش می‌آید که منظره یک دره‌ی دراز و صاف داشته باشد (دو پیچ می‌توانند اثر هم را جبران کنند)، یک فلات صاف (کوهنورد نمی‌فهمد کدام طرف سرازیری است)، یا چند دره‌ی جدا (کوهنورد ممکن است در دره‌ی اشتباه بایستد). بیشتر این هفته درباره‌ی تشخیص این موقعیت‌ها و محافظت از خودمان در برابر آن‌هاست.

**زمان‌بندی پیشنهادی (۶۰ دقیقه):** توابع ناهمخوانی ۱۰ · سطح خطا ۱۰ · الگوریتم‌های بهینه‌سازی ۲۰ · مشکلات عملی و چک‌لیست برازش ۱۵ · پیش‌نمایش ۵.

### ۱. توابع ناهمخوانی

برای مشاهدات $y_i$ و پیش‌بینی‌های مدل $\hat{y}_i = f(x_i;\boldsymbol{\theta})$، $i = 1,\dots,N$:

$$\mathrm{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \big(y_i - f(x_i;\boldsymbol{\theta})\big)^2, \qquad \mathrm{MSE} = \frac{\mathrm{SSE}}{N}, \qquad \mathrm{RMSD} = \sqrt{\mathrm{MSE}}.$$

- هر سه **کمینه‌کننده‌ی یکسانی** دارند: تبدیل‌های یکنوای همدیگرند. SSE را کمینه و RMSD را گزارش می‌کنیم، چون RMSD بر حسب واحد داده است (اینجا: نسبت یادآوری)، پس «RMSD = ۰.۰۵۷» یعنی پیش‌بینی‌ها به‌طور میانگین حدود ۰.۰۶ خطا دارند.
- **برآورد کمترین مربعات** این است:

$$\hat{\boldsymbol{\theta}}_{\mathrm{LS}} = \arg\min_{\boldsymbol{\theta}} \mathrm{SSE}(\boldsymbol{\theta}).$$

**چرا مربع؟** مربع کردن همه‌ی خطاها را مثبت می‌کند، خطاهای بزرگ را به‌شدت جریمه می‌کند، و تابعی هموار (مشتق‌پذیر) می‌دهد که بهینه‌سازها راحت با آن کار می‌کنند. در سطحی عمیق‌تر، کمینه کردن SSE دقیقاً همان برآورد بیشینه درست‌نمایی است، **اگر** نویز گاوسی و با واریانس یکسان برای همه‌ی مشاهدات باشد (هفته‌ی ۴). این «اگر» فرض پنهان کمترین مربعات است.

**وقتی فرض برقرار نیست.** نسبت‌های یادآوری دوجمله‌ای‌اند: واریانسشان $f(1-f)/n$ به $f$ بستگی دارد. نقاط نزدیک ۰.۵ پرنویزتر از نقاط نزدیک ۰ یا ۱ هستند، ولی SSE ساده با همه‌شان مثل هم رفتار می‌کند. **کمترین مربعات وزنی** (weighted least squares) این را با وزن دادن به هر مربع خطا بر اساس معکوس واریانسش درست می‌کند:

$$\mathrm{WSSE}(\boldsymbol{\theta}) = \sum_{i} \frac{\big(y_i - \hat{y}_i\big)^2}{\mathrm{Var}(y_i)}.$$

این را در تمرین ۳ امتحان می‌کنید. هفته‌ی بعد، بیشینه درست‌نمایی این مسئله را طبیعی‌تر حل می‌کند.

> **نکته:** بعضی کتاب‌ها به RMSD می‌گویند «RMSE». هر دو یک کمیت‌اند.

#### خواندن ریاضی

- $\sum_{i=1}^{N}$ یعنی «روی همه‌ی $N$ نقطه‌ی داده جمع بزن». $y_i - f(x_i;\boldsymbol{\theta})$ **باقی‌مانده** (residual) نقطه‌ی $i$ است: مقدار مشاهده‌شده منهای مقدار پیش‌بینی‌شده. مربع کردن و جمع زدن برای هر تنظیم پارامتر $\boldsymbol{\theta}$ یک عدد می‌دهد.
- $\arg\min_{\boldsymbol{\theta}} \mathrm{SSE}(\boldsymbol{\theta})$ این‌طور خوانده می‌شود: «مقداری از $\boldsymbol{\theta}$ که در آن SSE کمترین است». به تفاوت دقت کنید: $\min$ کوچک‌ترین مقدار *SSE* است و $\arg\min$ مقدار پارامتری که به آن می‌رسد. ما $\arg\min$ را می‌خواهیم.
- **مثال کوچک.** داده‌ی $y = (0.62, 0.41)$ و پیش‌بینی $(0.66, 0.44)$: باقی‌مانده‌ها $(-0.04, -0.03)$، $\mathrm{SSE} = 0.0016 + 0.0009 = 0.0025$، $\mathrm{MSE} = 0.00125$ و $\mathrm{RMSD} \approx 0.035$: منحنی به‌طور میانگین حدود ۳.۵ درصد از نقطه‌ها فاصله دارد.
- در WSSE هر مربع باقی‌مانده بر واریانس همان نقطه تقسیم می‌شود، پس نقطه‌ی پرنویز وزن کمتری دارد. برای داده‌ی دوجمله‌ای واریانس برابر $f(1-f)/n$ است.

### ۲. سطح خطا (error surface)

برای مدلی با $k$ پارامتر، $\mathrm{SSE}(\boldsymbol{\theta})$ یک سطح روی فضای پارامتر $k$ بعدی است. با $k = 2$ می‌توانیم آن را به شکل نقشه‌ی تراز (contour map) رسم کنیم: مسئله‌ی برازش می‌شود «پایین‌ترین نقطه‌ی این منظره را پیدا کن.»

ویژگی‌هایی که باید بشناسید:

| ویژگی | ظاهرش | معنایش |
|---|---|---|
| **یک کاسه‌ی گرد** | خطوط تراز هم‌مرکز و تقریباً دایره‌ای | پارامترها به‌خوبی تعیین می‌شوند و تقریباً مستقل‌اند؛ هر الگوریتم معقولی کمینه را پیدا می‌کند |
| **دره‌ی بلند و مورب** | خطوط تراز کشیده و کج | trade-off **پارامترها**: افزایش یک پارامتر را می‌شود با تغییر دیگری جبران کرد. برآوردها در امتداد دره همبسته و نامطمئن خواهند بود (هفته‌ی ۵، هفته‌ی ۷) |
| **فلات** (plateau) | خطوط تراز خیلی دور از هم | پیش‌بینی‌های مدل با پارامترها تقریباً تغییر نمی‌کنند (مثلاً نرخ زوال آن‌قدر بزرگ که منحنی پیش‌بینی‌شده همه‌جا ۰ است). روش‌های گرادیانی گیر می‌کنند |
| **چند حوضه‌ی جدا** | بیش از یک مجموعه‌ی بسته از خطوط تراز | **کمینه‌های محلی**: جواب الگوریتم به نقطه‌ی شروع بستگی دارد |

در فراموشی نمایی، $A$ و $\lambda$ با هم trade-off دارند: سطح شروع بالاتر با زوال سریع‌تر می‌تواند تقریباً همان مقادیر میانی را پیش‌بینی کند که سطح پایین‌تر با زوال کندتر. به همین دلیل دره مورب است (در lab می‌بینید).

## ۳. الگوریتم‌های بهینه‌سازی

همه‌ی الگوریتم‌های زیر از مقادیر اولیه‌ی $\boldsymbol{\theta}^{(0)}$ شروع می‌کنند و دنباله‌ای $\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}, \ldots$ می‌سازند که (امیدواریم) به کمینه پایین برود.

### ۱.۳ جستجوی شبکه‌ای (grid search)

SSE را در همه‌ی نقاط یک شبکه‌ی منظم حساب کنید و بهترین را نگه دارید.

- **مزایا:** فوق‌العاده ساده؛ گول کمینه‌های محلی روی شبکه را نمی‌خورد؛ تصویری از کل سطح می‌دهد.
- **معایب:** دقت محدود به فاصله‌ی نقاط شبکه است؛ هزینه با تعداد پارامترها **به‌طور نمایی** بالا می‌رود. با ۱۰۰ مقدار برای هر پارامتر: $k = 2$ به $10^4$ ارزیابی و $k = 5$ به $10^{10}$ ارزیابی نیاز دارد. این همان **نفرین ابعاد** (curse of dimensionality) است.
- **کاربرد:** رسم سطح‌های ۱ یا ۲ پارامتری، و تولید مقادیر شروع خوب.

### ۲.۳ سیمپلکس Nelder–Mead (بدون مشتق)

یک **سیمپلکس** مجموعه‌ای از $k+1$ نقطه در فضای $k$ بعدی است (برای $k = 2$ یک مثلث). هر تکرار سعی می‌کند **بدترین** رأس $\boldsymbol{\theta}_w$ را با حرکت دادنش از میان مرکز ثقل $\bar{\boldsymbol{\theta}}$ بقیه‌ی رأس‌ها جایگزین کند:

| حرکت | فرمول (ضرایب استاندارد) | چه وقت |
|---|---|---|
| بازتاب (reflect) | $\boldsymbol{\theta}_r = \bar{\boldsymbol{\theta}} + (\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_w)$ | همیشه اول امتحان می‌شود |
| گسترش (expand) | $\boldsymbol{\theta}_e = \bar{\boldsymbol{\theta}} + 2(\bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_w)$ | نقطه‌ی بازتاب‌شده از بهترین فعلی بهتر است؛ جلوتر برو |

| | حرکت | فرمول (ضرایب استاندارد) | چه وقت |
|---|---|---|---|
| انقباض (contraction) | همین گام در سمت بازتاب (خارجی) | $\boldsymbol{\theta}_c = \bar{\boldsymbol{\theta}} + \frac{1}{2}(\boldsymbol{\theta}_w - \bar{\boldsymbol{\theta}})$ (داخلی)، یا | نقطه‌ی بازتاب‌شده از دومین بدترین رأس بهتر نیست |
| جمع‌شدن (shrink) | همه‌ی رأس‌ها را تا نیمه به سمت بهترین ببر | | انقباض هم شکست خورد |

نقطه‌ی بازتاب‌شده **پذیرفته می‌شود** وقتی از دومین بدترین رأس بهتر باشد ولی از بهترین بهتر نباشد. سیمپلکس سرازیر «غلت می‌خورد»، در امتداد دره‌ها کشیده می‌شود و دور کمینه جمع می‌شود.

- **مزایا:** فقط به مقادیر SSE نیاز دارد (بدون مشتق)؛ در برابر توابع هدف پرنویز یا ناهموار مقاوم است؛ ابزار پیش‌فرض و پرکاربرد در مدل‌سازی شناختی.
- **معایب:** در ابعاد بالا ممکن است کند باشد؛ در دره‌های بلند و باریک ممکن است گیر کند؛ هیچ تضمینی برای پیدا کردن کمینه‌ی سراسری ندارد.

روش‌های جستجوی مستقیم مثل Nelder–Mead در کتاب قدوسیان درباره‌ی روش‌های مستقیم در بهینه‌سازی غیرخطی به‌تفصیل بررسی شده‌اند (مطالعه‌ی بیشتر را ببینید).

**۳.۳ روش‌های گرادیانی** **گرادیان** $\nabla \mathrm{SSE}(\boldsymbol{\theta})$ به سمت سربالایی اشاره می‌کند، پس ما خلافش گام برمی‌داریم:

$$\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta \, \nabla \mathrm{SSE}\big(\boldsymbol{\theta}^{(m)}\big), \qquad \frac{\partial \,\mathrm{SSE}}{\partial \theta} = -2 \sum_i \big(y_i - \hat{y}_i\big) \, \frac{\partial \hat{y}_i}{\partial \theta},$$

با اندازه‌ی گام (نرخ یادگیری) $\eta$. برای مدل نمایی، $\partial \hat{y}/\partial A = e^{-\lambda t}$ و $\partial \hat{y}/\partial \lambda = -A\, t\, e^{-\lambda t}$.

در عمل از روش‌های **شبه‌نیوتنی** (quasi-Newton) مثل BFGS استفاده می‌کنیم. این روش‌ها انحنای سطح را هم تخمین می‌زنند و اندازه‌ی گام را خودکار انتخاب می‌کنند، پس نزدیک کمینه خیلی سریع‌تر همگرا می‌شوند. L-BFGS-B نسخه‌ای است که **کران** پارامترها را می‌پذیرد. اگر گرادیان را ندهید، scipy آن را عددی با تفاضل‌های متناهی تقریب می‌زند.

- **مزایا:** روی سطوح هموار سریع و دقیق‌اند؛ برای تعداد پارامتر زیاد خوب مقیاس‌پذیرند (شبکه‌های عصبی همین‌طور آموزش می‌بینند).
- **معایب:** به تابع هدف هموار نیاز دارند؛ هر جا گرادیان (تقریباً) صفر باشد گیر می‌کنند، هم در فلات‌ها و هم در کمینه‌ها؛ به مقدار شروع حساس‌اند.

**پیشرفته: چرا فلات‌ها روش‌های گرادیانی را به دام می‌اندازند.** اگر $\lambda$ خیلی بزرگ باشد، $e^{-\lambda t} \approx 0$ در همه‌ی $t$های مشاهده‌شده، پس هر دو مشتق جزئی بالا $0 \approx$ هستند. روش گرادیانی یک «کف صاف» می‌بیند و متوقف می‌شود و گاهی همگرایی موفق گزارش می‌کند. Nelder–Mead مقادیر تابع را در نقاط دور مقایسه می‌کند و می‌تواند از فلات بیرون بیاید. هر دو رفتار را در بخش D lab می‌بینید.

**خلاصه:**

| روش | نیاز به مشتق | کران | کاربرد معمول |
|---|---|---|---|
| جستجوی شبکه‌ای | ندارد | ذاتی | رسم، مقدار شروع، $k \le 2$ |
| Nelder–Mead | ندارد | دارد (scipy ۷.۱ $\ge$) | پیش‌فرض برای مدل‌های شناختی کوچک؛ توابع هدف پرنویز |
| BFGS | عددی یا تحلیلی | ندارد | توابع هدف هموار، پارامترهای بدون قید |
| L-BFGS-B | عددی یا تحلیلی | دارد | توابع هدف هموار با پارامترهای کران‌دار |

**خواندن ریاضی** گام گرادیانی $\boldsymbol{\theta}^{(m+1)} = \boldsymbol{\theta}^{(m)} - \eta\,\nabla\mathrm{SSE}(\boldsymbol{\theta}^{(m)})$ این‌طور خوانده می‌شود: حدس جدید = حدس قبلی − اندازه‌ی گام × شیب. بالانویس $(m)$ شماره‌ی تکرار است، نه توان. $\nabla\mathrm{SSE}$ بردار مشتق‌های جزئی است (برای هر پارامتر یک شیب). علامت منفی گام را به سمت سرازیری می‌برد.

فرمول مشتق $\partial\,\mathrm{SSE}/\partial\theta = -2\sum_i (y_i - \hat{y}_i)\,\partial\hat{y}_i/\partial\theta$ از قاعده‌ی زنجیره‌ای می‌آید. می‌گوید: نقطه‌هایی که باقی‌مانده‌شان بزرگ است و پیش‌بینی در آن‌ها به $\theta$ حساس است، $\theta$ را قوی‌تر از بقیه می‌کشند. در نقطه‌ی کمینه این کشش‌ها همدیگر را خنثی می‌کنند و جمع صفر می‌شود.

### برای یک مهندس

- **کمترین مربعات** همان معیار تخمین کانال و برازش منحنی است؛ تفاوت اینجا این است که مدل‌ها نسبت به پارامترها **غیرخطی**‌اند ($e^{-\lambda t}$)، پس جواب بسته‌ای از معادله‌های نرمال وجود ندارد و به جست‌وجوی تکراری نیاز داریم.
- **کمترین مربعات وزنی** همان سفید کردن نویز (noise whitening) است: تقسیم بر واریانس همان کاری است که وقتی نویز سفید نیست (نویز ناهمسان‌واریانس) پیش از LS انجام می‌دهید.
- **گرادیان نزولی با اندازه‌ی گام** $\eta$ همان ایده‌ی steepest descent در فیلتر تطبیقی LMS است. BFGS یک روش شبه‌نیوتنی است (تقریبی از Hessian می‌سازد)، نزدیک به Gauss–Newton / Levenberg–Marquardt در LS غیرخطی.
- **یک دره‌ی دراز مورب** در سطح خطا یعنی مسئله **بدحالت** (ill-conditioned) است: نسبت مقدار ویژه‌های Hessian بزرگ است، مثل وقتی که رگرسورها به‌شدت همبسته‌اند. پارامترها در یک جهت خوب و در جهت دیگر بد تعیین می‌شوند.
- reparameterization ($\lambda = e^{\eta}$) یک تغییر متغیر است که مسئله‌ی مقید را به مسئله‌ی بدون قید تبدیل می‌کند و اغلب حالت (conditioning) مسئله را هم بهتر می‌کند.
- **تفاوت با کار مهندسی:** مجموعه داده‌ها کوچک‌اند (۸ نقطه، هرکدام ۵۰ آیتم)، نویز زیاد است، و پارامترها باید از نظر روان‌شناختی تفسیر شوند. پس کافی نیست منحنی پیش‌بینی‌شده خوب به نظر برسد؛ تک‌تک مقدارهای پارامتر باید قابل اعتماد باشند.

## ۴. مشکلات عملی و راه‌حل‌ها

**۱. مقادیر شروع.** از شروع‌های آگاهانه استفاده کنید: از روی نمودار بخوانید، یک برازش خطی‌شده‌ی سریع (هفته‌ی ۲) بگیرید، یا بهترین نقطه‌ی یک شبکه‌ی درشت را بردارید.

**۲. کمینه‌های محلی** ← multi-start. بهینه‌ساز را از تعداد زیادی نقطه‌ی شروع تصادفی اجرا کنید و بهترین جواب را نگه دارید. اگر شروع‌های زیادی روی یک کمینه توافق کنند، می‌توانید نسبتاً مطمئن باشید که کمینه‌ی سراسری است. multi-start بیمه‌ی ارزانی است؛ برای مدل‌های بیش از دو پارامتر همیشه از آن استفاده کنید.

**۳. کران پارامترها.** بسیاری از پارامترها دامنه‌ی طبیعی دارند ($0 \le A \le 1$، $\lambda > 0$). بدون قید، بهینه‌سازها با خیال راحت $A = 29.6$ برمی‌گردانند. سه رویکرد:

| رویکرد | چطور | توضیح |
|---|---|---|
| بهینه‌ساز کران‌دار | `method="L-BFGS-B"` یا `"Nelder-Mead"` با `bounds=` | ساده‌ترین؛ پیش‌فرض توصیه‌شده |
| reparameterization | روی $\eta$ بدون قید بهینه کنید و تبدیل کنید: $\lambda = e^{\eta}$ (مثبت)، $A = 1/(1+e^{-\alpha_A})$ (بین ۰ و ۱) | با هر بهینه‌سازی کار می‌کند؛ در مدل‌سازی بیزی هم استفاده می‌شود (هفته‌ی ۱۳) |
| جریمه | وقتی پارامتر خارج از دامنه است یک SSE خیلی بزرگ برگردانید | ناپیوستگی می‌سازد؛ از آن پرهیز کنید |

**۴.** non-identifiability. گاهی چند ترکیب پارامتر به یک اندازه خوب برازش می‌شوند. مثالش مدلی دوجزئی است که برچسب دو جزئش قابل جابه‌جایی است. این ویژگی خود مدل است، نه شکست بهینه‌ساز، و multi-start آن را آشکار می‌کند (بخش D lab).

**۵. اعتماد کن، ولی بررسی کن.** `result.success==True` فقط یعنی قاعده‌ی توقف الگوریتم برقرار شده است. همیشه (۱) منحنی برازش‌شده را روی داده رسم کنید، (۲) به باقی‌مانده‌ها نگاه کنید، و (۳) بررسی کنید برآوردها در دامنه‌ی معقول باشند.

**چک‌لیست برازش (از این به بعد در هر lab از آن استفاده کنید):**

۱. داده‌ها را رسم کنید.
۲. مدل و تابع ناهمخوانی را بنویسید؛ آن‌ها را روی چند مقدار پارامتر دلخواه آزمون کنید.
۳. کران‌ها و مقادیر شروع معقول انتخاب کنید.
۴. بهینه کنید، ترجیحاً با multi-start.
۵. پیام همگرایی، نمودار، باقی‌مانده‌ها و دامنه‌ی پارامترها را بررسی کنید.
۶. یک بررسی بازیابی انجام دهید: از پارامترهای برازش‌شده داده شبیه‌سازی کنید، دوباره برازش کنید و مقایسه کنید (هفته‌ی ۷ این را نظام‌مند می‌کند).

> **پیشرفته: پارامترهای شرطاً خطی.** در $f(t) = A\,e^{-\lambda t}$، برای هر $\lambda$ ثابت بهترین $A$ فرم بسته دارد، چون مدل نسبت به $A$ خطی است: $\hat{A}(\lambda) = \sum_i y_i g_i / \sum_i g_i^2$ با $g_i = e^{-\lambda t_i}$. جای‌گذاری آن مسئله را به یک مسئله‌ی یک‌بعدی فقط بر حسب $\lambda$ تبدیل می‌کند. این ترفند (حذف پارامترهای خطی یا ***variable projection***) برازش را سریع‌تر و پایدارتر می‌کند، و تابع یک‌بعدی حاصل با ***profile likelihood*** هفته‌ی ۵ مرتبط است.

**برای مدرس**

- **اول منظره را نشان دهید.** نمودار تراز SSE روی $(A, \lambda)$ (بخش B lab) همه‌ی ایده‌های بعدی را ملموس می‌کند: کمینه، دره‌ی مورب، فلات. آن را زود روی پروژکتور بیاورید و مدام به آن برگردید.
- **Nelder–Mead را اجرا کنید.** یک مثلث روی نمودار تراز بکشید و یک بازتاب (reflection) را با دست انجام دهید (قدم ۲ مثال حل‌شده). دانشجوها «مثلثی که قل می‌خورد» را خیلی بهتر از جدول حرکت‌ها به خاطر می‌سپارند.

- **نقطه‌ی قوت شما:** شما gradient descent و کمترین مربعات را می‌شناسید. وقت کمتری روی الگوریتم‌ها و وقت بیشتری روی حالت‌های شکست بگذارید (بخش D lab: بهینه‌سازی که در $A = 29.6$ اعلام موفقیت می‌کند، به‌یادماندنی‌ترین درس این هفته است.
- **سؤال‌های رایج.**
  - «از کدام بهینه‌ساز استفاده کنم؟» برای مدل‌های شناختی کوچک: Nelder–Mead یا L-BFGS-B با کران، همیشه همراه با multi-start.
  - «چرا همیشه از *grid* استفاده نکنیم؟» هزینه با تعداد پارامترها به‌طور نمایی زیاد می‌شود.
  - «آیا *SSE* معیار درستی است؟» فقط اگر نویز گاوسی و با واریانس برابر باشد؛ هفته‌ی بعد درست‌نمایی جای آن را می‌گیرد.

---

### ۵.۱.۱ مثال حل‌شده

**داده.** یک شرکت‌کننده نسبت‌های $y = (0.62, 0.41, 0.19)$ را در $t = (1, 3, 7)$ روز به یاد می‌آورد. مدل: $f(t) = A\,e^{-\lambda t}$.

#### قدم ۱: یک جستجوی شبکه‌ای کوچک با دست

برای $(A, \lambda) = (0.8, 0.2)$: پیش‌بینی‌ها $0.8e^{-0.2} = 0.655$، $0.8e^{-0.6} = 0.439$، $0.8e^{-1.4} = 0.197$، پس

$$\text{SSE} = (0.62-0.655)^2 + (0.41-0.439)^2 + (0.19-0.197)^2 = 0.00123 + 0.00084 + 0.00005 \approx 0.00212.$$

با تکرار این کار برای یک شبکه‌ی ۲ × ۳:

| | $\lambda = 0.15$ | $\lambda = 0.20$ | $\lambda = 0.25$ |
|---|---|---|---|
| $A = 0.8$ | ۰۲۲۸۱.۰ | **۰۰۲۱۲.۰** | ۰۰۳۶۴.۰ |
| $A = 0.9$ | ۰۶۶۳۸.۰ | ۰۲۱۷۲.۰ | ۰۰۷۹۱.۰ |

بهترین نقطه‌ی شبکه $(0.8, 0.20)$ است. به کجی دقت کنید: $(0.9, 0.25)$ خیلی بهتر از $(0.9, 0.15)$ است. $A$ بزرگ‌تر یک $\lambda$ بزرگ‌تر «می‌خواهد»؛ این همان دره‌ی trade-off است.

#### قدم ۲: یک تکرار Nelder–Mead با دست

با سیمپلکس (مثلث) سه نقطه‌ی شبکه شروع کنید:

| رأس | $(A, \lambda)$ | SSE | رتبه |
|---|---|---|---|
| $\boldsymbol{\theta}_b$ | (۰.۸، ۰.۲۵) | ۰۰۳۶۴.۰ | بهترین |
| $\boldsymbol{\theta}_g$ | (۰.۸، ۰.۱۵) | ۰۲۲۸۱.۰ | دومین بدترین |
| $\boldsymbol{\theta}_w$ | (۰.۹، ۰.۱۵) | ۰۶۶۳۸.۰ | بدترین |

۱. **مرکز ثقل** همه‌ی رأس‌ها جز بدترین: $\bar{\boldsymbol{\theta}} = \big((0.8+0.8)/2,\ (0.25+0.15)/2\big) = (0.8, 0.20)$.
۲. **بازتاب** بدترین رأس از میان مرکز ثقل: $\boldsymbol{\theta}_r = (0.8, 0.20) + \big((0.8, 0.20) - (0.9, 0.15)\big) = (0.7, 0.25)$.
۳. **ارزیابی:** پیش‌بینی‌ها $(0.545, 0.331, 0.122)$، پس $\mathrm{SSE}(\boldsymbol{\theta}_r) = 0.01657$.
۴. **تصمیم:** $0.00364 \le 0.01657 < 0.02281$. نقطه‌ی بازتاب‌شده از دومین بدترین بهتر است ولی از بهترین بهتر نیست، پس **بازتاب را می‌پذیریم** (بدون گسترش).
۵. **سیمپلکس جدید:** $\{(0.8, 0.25),\ (0.7, 0.25),\ (0.8, 0.15)\}$، و بدترین SSE از ۰۶۶.۰ به ۰۲۳.۰ کم شده است.

#### قدم ۳: بگذارید کامپیوتر تمامش کند

اجرای `scipy.optimize.minimize(...,method="Nelder-Mead")` از $(0.8, 0.2)$ به $\hat{A} = 0.754$، $\hat{\lambda} = 0.200$ با $\mathrm{RMSD\ SSE} = 3.7 \times 10^{-5}$ ($= 0.0035$) همگرا می‌شود.

**تفسیر.** مدل برازش‌شده می‌گوید این شرکت‌کننده بلافاصله بعد از یادگیری حدود ۷۵٪ را به یاد می‌آورد و هر روز حدود ۱۸٪ از باقی‌مانده را از دست می‌دهد ($1 - e^{-0.2} = 0.181$)، با نیمه‌عمر $\ln 2 / 0.2 \approx 3.5$ روز. با فقط سه داده و دو پارامتر، برازش تقریباً کامل چیز زیادی درباره‌ی درست بودن مدل نمایی نمی‌گوید. هر منحنی نزولی دوپارامتری معقولی سه نقطه را خوب برازش می‌کند (هفته‌ی ۶).

---

### ۶.۱.۱ lab (۲ ساعت)

#### اهداف

۱. نوشتن یک تابع SSE و بررسی سطح خطای دو مدل فراموشی.
۲. برازش با جستجوی شبکه‌ای، سپس با Nelder–Mead و L-BFGS-B؛ مقایسه با برازش‌های خطی‌شده‌ی هفته‌ی قبل.
۳. ایجاد و رفع شکست‌های برازش: فلات‌ها، کمینه‌های محلی و برآوردهای خارج از دامنه.
۴. **ارزیابی** برازش‌ها (RMSD، باقی‌مانده‌ها، مقایسه با مقادیر واقعی) و **تفسیر** پارامترها.

**زمان‌بندی:** بخش A ۱۵ دقیقه · بخش B ۲۵ دقیقه · بخش C ۲۵ دقیقه · بخش D ۳۵ دقیقه · بخش E ۲۰ دقیقه.

#### آماده‌سازی و داده

Code · **کد:** `code/week03.py`، cell ۱ · سلول ۱

#### بخش A: آشنایی با تابع ناهمخوانی (۱۵ دقیقه)

Code · **کد:** `code/week03.py`، cell ۲ · سلول ۲

**خروجی مورد انتظار:**

```
theta = (0.9, 0.5):  power SSE = 0.0349   exponential SSE = 0.4511
theta = (0.8, 0.4):  power SSE = 0.0328   exponential SSE = 0.4145
theta = (0.9, 0.1):  power SSE = 0.9047   exponential SSE = 0.2192
theta = (0.6, 0.05):  power SSE = 0.3303   exponential SSE = 0.1145
```

پیش از اجرای بخش‌های بعد **پیش‌بینی کنید**: کدام مقادیر پارامتر کمترین SSE را برای هر مدل می‌دهند؟ حدستان را یادداشت کنید.

### بخش B: جستجوی شبکه‌ای و سطح خطا (۲۵ دقیقه)

Code · **کد:** `code/week03.py`، cell ۳ · سلول ۳

**خروجی مورد انتظار:**

```
exp: grid minimum A = 0.655, rate = 0.0560, SSE = 0.10607  (42300
↪  evaluations)
pow: grid minimum A = 0.870, rate = 0.4239, SSE = 0.02592  (42300
↪  evaluations)
```

**به این دقت کنید:** هر دو سطح یک حوضه دارند، ولی یک **دره‌ی کشیده و مورب**: $A$ بالاتر با زوال بیشتر همراه است. این همان trade-off پارامترهاست. برای مدل توانی، مقدار واقعی (× سفید) داخل دره است ولی در ته آن نیست: با داده‌ی محدود، بهترین برازش هیچ‌وقت دقیقاً همان مقدار واقعی نیست.

### بخش C: برازش با یک بهینه‌ساز (۲۵ دقیقه)

Code · **کد:** `code/week03.py`، cell ۴ · سلول ۴

**خروجی مورد انتظار:**

```
model            method      A   rate    SSE   RMSD  f_evals success
  exp         L-BFGS-B 0.6568 0.0566 0.1061 0.1151     39.0    True
  exp       Nelder-Mead 0.6568 0.0566 0.1061 0.1151     69.0    True
  exp              grid 0.6550 0.0560 0.1061 0.1151  42300.0    True
  exp linearized (Wk 2) 0.5500 0.0320 0.1373 0.1310      NaN     NaN
  pow         L-BFGS-B 0.8712 0.4251 0.0259 0.0569     36.0    True
  pow       Nelder-Mead 0.8711 0.4251 0.0259 0.0569     71.0    True
  pow              grid 0.8700 0.4239 0.0259 0.0569  42300.0    True
  pow linearized (Wk 2) 0.7970 0.3700 0.0315 0.0628      NaN     NaN
```

Code · **کد:** `code/week03.py`، cell ۵ · سلول ۵

**به این‌ها دقت کنید:** – Nelder–Mead و L-BFGS-B تا سه یا چهار رقم اعشار با هم توافق دارند؛ شبکه در حد فاصله‌ی نقاطش (۰۰۵.۰ برای $A$) با آن‌ها فرق دارد. بهینه‌سازها فقط ۳۶ تا ۷۱ ارزیابی تابع لازم داشتند، به‌جای ۴۲ ۳۰۰. – کمترین مربعات SSE **کمتری** از برازش‌های خطی‌شده‌ی هفته‌ی ۲ می‌دهد (دقیقاً همین کمیت را کمینه می‌کند)، و برآوردهای مدل توانی خیلی به مقدار واقعی نزدیک‌ترند. – مسیر اول سریع وارد دره می‌شود و بعد آرام در امتدادش می‌خزد؛ یک الگوی معمول.

### بخش D: وقتی برازش خراب می‌شود (۳۵ دقیقه)

D1. **یک فلات و یک «موفقیت» کاذب.** برازش نمایی را با یک نرخ زوال بی‌معنی، $\lambda = 20$ در روز، شروع کنید:

Code · **کد:** `code/week03.py`، cell ۶ · سلول ۶

**خروجی مورد انتظار:**

```
BFGS         A =   29.571  lambda =  14.244  SSE = 1.1616  success =
 ↪  True  (Optimization terminated successfully.)
L-BFGS-B     A =   29.905  lambda =  14.289  SSE = 1.1616  success =
 ↪  True  (CONVERGENCE: NORM_OF_PROJECTED_GRADIENT_<=_PGTOL)
Nelder-Mead  A =    0.657  lambda =   0.057  SSE = 0.1061  success =
 ↪  True  (Optimization terminated successfully.)
```

(متن پیام L-BFGS-B بین نسخه‌های SciPy کمی فرق می‌کند.) دو روش گرادیانی **موفقیت** گزارش می‌کنند ولی برازشی بی‌معنی برمی‌گردانند: $A$ حدود ۳۰ است (احتمال یادآوری ٪۳۰۰۰!) و منحنی پیش‌بینی‌شده عملاً در همه‌ی فاصله‌ها صفر است. آن‌ها روی فلاتی شروع کردند که گرادیان در آن تقریباً صفر است (کادر «پیشرفته» بخش ۳.۳ جزوه). Nelder–Mead فرار کرد. درس‌ها: (۱) همیشه برآوردها و نمودارها را بررسی کنید، نه فقط `success` را؛ (۲) از کران استفاده کنید؛ (۳) از مقادیر شروع معقول یا متعدد استفاده کنید.

D2. reparameterization. پارامترها را بدون قید کنید تا هر بهینه‌سازی دامنه‌های $0 < A < 1$ و $\lambda > 0$ را رعایت کند:

Code · **کد:** `code/week03.py`، cell ۷ · سلول ۷

**خروجی مورد انتظار:**

```
BFGS on reparameterized power model: A = 0.8712, d = 0.4251, SSE =
↪  0.02592
```

همان جواب بخش C، و تضمین می‌شود که برآوردها در دامنه باشند.

D3. **کمینه‌های محلی و** multi-start. یک ایده‌ی رایج در پژوهش‌های حافظه این است که فراموشی بازتاب دو رد حافظه (memory trace) است: یک جزء **سریع** که زود زوال پیدا می‌کند و یک جزء **کند** که می‌ماند:

$$f(t) = A_1 e^{-\lambda_1 t} + A_2 e^{-\lambda_2 t}.$$

این مدل $k = 4$ پارامتر دارد و سطح خطایش چند حوضه دارد. آن را از ۵۰ نقطه‌ی شروع تصادفی برازش کنید:

Code · **کد:** `code/week03.py`، cell ۸ · سلول ۸

**خروجی مورد انتظار:**

```
Distinct final SSE values (rounded) and how often each was reached:
  SSE = 0.0201: 42 starts
  SSE = 0.0203: 1 starts
  SSE = 0.0337: 1 starts
  SSE = 0.0340: 1 starts
  SSE = 0.0359: 1 starts
  SSE = 0.1064: 1 starts
  SSE = 0.2010: 1 starts
  SSE = 0.2456: 1 starts
  SSE = 0.9092: 1 starts
Best 3 solutions (A1, lam1, A2, lam2):
   [0.58  0.36  0.264 0.   ] SSE = 0.02013
   [0.264 0.    0.58  0.36 ] SSE = 0.02013
   [0.264 0.    0.58  0.36 ] SSE = 0.02013
```

**به این‌ها دقت کنید:** - حدود ۴۰ شروع از ۵۰ در همان بهترین حوضه تمام می‌شوند، ولی بقیه (حدود ۱ از ۶) در **کمینه‌های محلی بدتر** گیر می‌کنند؛ بعضی با SSE چند برابر بزرگ‌تر. (تعداد دقیق بین نسخه‌های SciPy کمی فرق دارد؛ با SciPy ۱۸.۱ برابر ۴۱ از ۵۰ است.) یک اجرای تنها از یک شروع بدشانس جواب غلطی می‌داد، بدون هیچ هشداری. - در میان بهترین جواب‌ها، اجزای سریع و کند ممکن است **با برچسب‌های جابه‌جا** ظاهر شوند ($A_1, \lambda_1 \leftrightarrow A_2, \lambda_2$). هر دو همان منحنی را توصیف می‌کنند. این یک non-identifiability ذاتی مدل است، و راه‌حل رایج اعمال قید ترتیبی مثل $\lambda_1 > \lambda_2$ است. - در بهترین جواب، یکی از اجزا نرخ دقیقاً ۰ دارد (یک جزء ثابت و بدون زوال به اندازه‌ی ۰.۲۶۴). پس مدل دوردی برازش‌شده به مدل نمایی با مجانب تمرین ۴ تبدیل شده، با همان SSE (۰.۰۲۰۱). آیا این یک «حافظه‌ی دائمی» معنادار است یا فقط پارامترهای اضافه نویز را جذب کرده‌اند؟ یادتان باشد داده‌ها از یک مدل توانی آمده‌اند. برازش به‌تنهایی نمی‌تواند جواب دهد (هفته‌های ۶–۷).

**بخش E: ارزیابی و تفسیر (۲۰ دقیقه)**

Code ۰ **کد:** `code/week03.py`، cell ۰۹ ۰ سلول ۹

**خروجی مورد انتظار:**

```
Power model: relative loss rate at t =  1 days: 0.213 per day
Power model: relative loss rate at t = 20 days: 0.020 per day
Exponential model: constant relative loss rate 0.057 per day
```

**تفسیر (برای هرکدام ۲ تا ۳ جمله در notebook بنویسید):**

۱. الگوهای باقی‌مانده‌ها را مقایسه کنید. کدام مدل الگوی نظام‌مند نشان می‌دهد، و آن الگو درباره‌ی شکل فراموشی در این داده‌ها چه می‌گوید؟
۲. نرخ نسبی فراموشی مدل توانی از حدود ۲۱.۰ در روز بعد از ۱ روز به حدود ۰۲.۰ در روز بعد از ۲۰ روز می‌رسد. به زبان ساده توضیح دهید این چه ادعایی درباره‌ی حافظه دارد.
۳. برآوردهای کمترین مربعات ($\hat{A} \approx 0.87$، $\hat{d} \approx 0.43$) از برآوردهای خطی‌شده‌ی هفته‌ی قبل به مقدار واقعی نزدیک‌ترند ولی هنوز با آن برابر نیستند. آیا این شکست روش است؟ (راهنمایی: اگر آزمایش را با یک نمونه‌ی تصادفی جدید تکرار می‌کردید چه می‌شد؟)

**تمرین‌ها (به ترتیب سختی)**

**تمرین ۱** ⋆ **(کمینه‌کننده‌ی یکسان).** مدل توانی را یک بار با کمینه کردن SSE، یک بار MSE و یک بار RMSD برازش کنید. نشان دهید برآوردها (در حد تلرانس بهینه‌ساز) یکسان‌اند. چرا باز هم ترجیح می‌دهیم RMSD را گزارش کنیم؟

**تمرین ۲** ⋆ **(ابینگهاوس با کمترین مربعات).** داده‌های savings ابینگهاوس از تمرین ۳ هفته‌ی ۲ (مقادیر با علامت [بررسی شود]) را برای هر دو مدل با کمترین مربعات دوباره برازش کنید. پارامترها و RMSD را با برازش‌های خطی‌شده‌ی هفته‌ی ۲ مقایسه کنید.

**تمرین ۳** ⋆⋆ **(کمترین مربعات وزنی).** نسبت‌های دوجمله‌ای واریانس $f(1-f)/n$ دارند. WSSE (بخش ۱ جزوه) را با استفاده از $f$ **پیش‌بینی‌شده** در واریانس پیاده‌سازی کنید، مدل توانی را دوباره برازش کنید و برآوردها را با کمترین مربعات ساده مقایسه کنید. کدام نقاط داده تأثیر بیشتری پیدا می‌کنند و کدام کمتر؟ (هفته‌ی بعد می‌بینید که بیشینه درست‌نمایی این وزن‌دهی را خودکار انجام می‌دهد.)

**تمرین ۴** ⋆⋆ **(پارامتر بیشتر، برازش بهتر؟).** مدل سه‌پارامتری **نمایی با مجانب** $f(t) = B + (A-B)e^{-\lambda t}$ را که در آن $B$ مجانب بلندمدت است، با کران‌های $0 \le B \le A \le 1$ برازش کنید (راهنمایی: reparameterize کنید، یا قید را بعد از برازش بررسی کنید). انتظار: SSE $\approx$ ۰۲۰.۰ که **کمتر** از SSE $\approx$ ۰۲۶.۰ مدل توانی است، با اینکه داده‌ها از مدل توانی تولید شده‌اند! این درباره‌ی استفاده از SSE برای انتخاب بین مدل‌هایی با تعداد پارامتر متفاوت چه می‌گوید؟ (در هفته‌های ۶–۷ جواب درست این را می‌دهیم.)

**تمرین ۵** ⋆⋆⋆ **(پیشرفته: Nelder–Mead از صفر).** Nelder–Mead را برای $k = 2$ با حرکت‌های بخش ۲.۳ جزوه (بازتاب، گسترش، انقباض داخلی/خارجی، جمع‌شدن) و یک قاعده‌ی توقف بر اساس پراکندگی مقادیر SSE در سیمپلکس پیاده‌سازی کنید. آن را روی مدل توانی از $(0.8, 0.2)$ اجرا کنید، سیمپلکس را هر چند تکرار یک بار روی سطح خطا رسم کنید، و نتیجه و تعداد ارزیابی‌های تابع را با `scipy` مقایسه کنید.

lab check-off

به مدرس یا دستیار آموزشی نشان دهید: (۱) نمودارهای سطح خطا با کمینه‌های شبکه (بخش B، (۲) جدول مقایسه از بخش C، (۳) هیستوگرام multi-start (بخش D3) با یک جمله درباره‌ی آنچه نشان می‌دهد، و (۴) جوابتان به سؤال تفسیری ۱ (بخش E).

---

### ۷.۱.۱ ارتباط با هوش مصنوعی و LLMها

**قانون‌های مقیاس** (scaling laws) **همان برازش منحنی‌اند.** یکی از تأثیرگذارترین نتایج هوش مصنوعی امروز این است که خطای آزمون (test loss) مدل‌های زبانی با اندازه‌ی مدل $N$، اندازه‌ی داده و محاسبات به‌طور هموار کم می‌شود و از یک **قانون توانی** با مجانب پیروی می‌کند: $L(N) = E + B\,N^{-\alpha}$ (Kaplan et al., ۲۰۲۰؛ Hoffmann et al., ۲۰۲۲). شرکت‌ها از چنین برازش‌هایی استفاده می‌کنند تا تصمیم بگیرند مدل بعدی چقدر بزرگ باشد. از نظر ریاضی، این همان مسئله‌ی برازش منحنی فراموشی است: یک مدل غیرخطی با سه پارامتر که با کمترین مربعات، یک بهینه‌ساز عددی و چند نقطه‌ی شروع برازش می‌شود. دام‌هایش هم همان‌اند: برون‌یابی (extrapolation) خیلی دورتر از داده، و trade-off پارامترها (اینجا $E$ و $\alpha$).

> **کد:** Code · سلول ۱۰ · cell ۱۰، `code/week03.py`

**خروجی مورد انتظار:**

```
E = 1.70, B = 3.22, alpha = 0.358, SSE = 0.00424
Extrapolated loss at N = 1e11: 1.754
```

**امتحانش کنید (بدون نیاز به API).** ⋆ - یک قانون توانی خالص بدون مجانب ($E = 0$) برازش کنید. SSE و برون‌یابی تا $N = 10^{11}$ را مقایسه کنید. کدام نتیجه‌گیری درباره‌ی «بزرگ‌تر بهتر است» عوض می‌شود؟ - ⋆⋆ فقط چهار مدل کوچک‌تر را برازش کنید و تا چهار مدل بزرگ‌تر برون‌یابی کنید. پیش‌بینی چقدر خطا دارد؟ با seedهای مختلف تکرار کنید تا ببینید برون‌یابی چقدر پایدار است. - ⋆⋆ سطح SSE را روی $E$ و $\alpha$ رسم کنید (با $B$ در بهترین مقدارش برای هر جفت). آیا مثل lab هفته‌ی ۳ یک ridge وجود دارد؟

مطالعه‌ی اختیاری: Kaplan, J., et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.
Hoffmann, J., et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.

---

### ۸.۱.۱ اشتباهات رایج و برداشت‌های نادرست

۱. **«`success=True` یعنی برازش خوب است.»** فقط یعنی یک قاعده‌ی توقف برقرار شده است (بخش D1). همیشه برآوردها، نمودارها و باقی‌مانده‌ها را وارسی کنید.
۲. **یک شروع کافی است.** برای هر مدلی فراتر از مدل‌های ساده‌ی دوپارامتری multi-start اجرا کنید. هزینه‌اش کم است؛ خطر گزارش یک کمینه‌ی محلی کم نیست.
۳. **نادیده گرفتن دامنه‌ی پارامترها.** بهینه‌سازهای بدون کران می‌توانند مقادیر ناممکن برگردانند (نرخ منفی، احتمال بیشتر از ۱). از کران یا reparameterization استفاده کنید.

۴. **مقایسه‌ی SSE بین مدل‌هایی با تعداد پارامتر متفاوت.** مدل منعطف‌تر تقریباً همیشه در SSE برنده است (تمرین ۴). این سؤالی درباره‌ی ارزیابی مدل است، نه برآورد (هفته‌های ۶–۷).
۵. **مقایسه‌ی SSE بین مجموعه‌داده‌هایی با اندازه‌ی متفاوت.** SSE با $N$ بزرگ می‌شود؛ برای مقایسه از RMSD استفاده کنید.
۶. **کمینه‌کننده را همان مقدار واقعی دانستن.** $\hat{\theta}$ برآوردی با خطای نمونه‌گیری است. نمونه‌ی دیگر برآورد دیگری می‌دهد (هفته‌ی ۵ این را کمّی می‌کند).
۷. **فراموش کردن فرض کمترین مربعات.** SSE ساده نویز گاوسی و برابر برای همه‌ی مشاهدات فرض می‌کند. نسبت‌ها، شمارش‌ها و زمان‌های واکنش معمولاً این فرض را نقض می‌کنند.
۸. **استفاده از جستجوی شبکه‌ای برای تعداد پارامتر زیاد.** هزینه منفجر می‌شود ($100^k$ ارزیابی). شبکه را فقط برای رسم و مقادیر شروع به کار ببرید.

---

### ۹.۱.۱ خودآزمایی

**۱.** چرا SSE، MSE و RMSD کمینه‌کننده‌ی یکسانی دارند؟ کدام را گزارش می‌کنید و چرا؟ **پاسخ** SSE/N = MSE و RMSD = $\sqrt{\text{MSE}}$ تبدیل‌های اکیداً صعودی SSE هستند (N ثابت است)، پس همان پارامترها هر سه را کمینه می‌کنند. RMSD را گزارش کنید، چون بر حسب واحد داده است و بین مجموعه‌داده‌هایی با اندازه‌ی متفاوت قابل مقایسه است.

**۲.** trade-off پارامترها چیست و روی سطح خطا چگونه دیده می‌شود؟ **پاسخ** تغییر یک پارامتر را می‌شود با تغییر پارامتر دیگری جبران کرد، طوری که پیش‌بینی‌ها (و SSE تقریباً عوض نشوند). روی سطح به شکل یک دره‌ی کشیده و مورب با SSE پایین دیده می‌شود. در این حالت برآوردها همبسته‌اند و در جهت دره ضعیف تعیین می‌شوند.

**۳.** با ۵۰ مقدار شبکه برای هر پارامتر، جستجوی شبکه‌ای برای یک مدل ۶ پارامتری به چند ارزیابی SSE نیاز دارد؟ این یعنی چه؟ **پاسخ** $50^6 \approx 1.56 \times 10^{10}$ ارزیابی. جستجوی شبکه‌ای فراتر از ۲ تا ۳ پارامتر عملی نیست (نفرین ابعاد)؛ به بهینه‌سازهای تکراری نیاز داریم.

**۴.** در یک تکرار Nelder–Mead، نقطه‌ی بازتاب‌شده از بهترین رأس فعلی بهتر است. الگوریتم بعد چه می‌کند؟ **پاسخ** یک گسترش امتحان می‌کند: نقطه‌ای در همان جهت با فاصله‌ی دو برابر از مرکز ثقل. اگر نقطه‌ی گسترش‌یافته از نقطه‌ی بازتاب‌شده بهتر باشد آن را نگه می‌دارد، وگرنه نقطه‌ی بازتاب‌شده را.

**۵.** چرا یک بهینه‌ساز گرادیانی ممکن است روی فلات متوقف شود و موفقیت گزارش کند؟ **پاسخ** قاعده‌ی توقفش بر اساس نزدیک به صفر بودن گرادیان (یا اندازه‌ی گام) است. روی فلات گرادیان نزدیک صفر است با اینکه آن نقطه کمینه نیست، پس معیار همگرایی برقرار می‌شود.

**۶.** دو راه برای مثبت نگه داشتن یک پارامتر نرخ زوال در حین برازش بگویید. **پاسخ** (۱) استفاده از بهینه‌ساز کران‌دار (مثلاً L-BFGS-B یا Nelder–Mead با `bounds=[(1e-6,None)]`). (۲) reparameterization: $\eta$ را روی کل اعداد حقیقی بهینه کنید و قرار دهید $\lambda = e^{\eta}$.

**۷.** برازش‌های multi-start مدل دوجزئی با $(A_1, \lambda_1, A_2, \lambda_2) = (0.58, 0.36, 0.26, 0.00)$ و $(0.26, 0.00, 0.58, 0.36)$ به یک SSE کمینه‌ی یکسان می‌رسند. آیا این مشکل بهینه‌ساز است؟ **پاسخ** نه. این جابه‌جایی برچسب (label switching) است: جابه‌جا کردن دو جزء دقیقاً همان منحنی را می‌دهد، پس مدل بدون یک قید ترتیبی (مثلاً $\lambda_1 > \lambda_2$) identifiable نیست. بهینه‌ساز به‌درستی دو کمینه‌ی سراسری معادل را پیدا کرده است.

---

### ۱۰.۱.۱ تکلیف

این هفته تکلیف نمره‌دار جدیدی منتشر نمی‌شود. A1 **(از کلامی به فرمال) در ابتدای درس هفته‌ی ۴ تحویل داده می‌شود.** lab امروز همه‌ی چیزهایی را که برای مرحله‌ی کالیبراسیون A1 لازم است به شما می‌دهد، اگر بخواهید استخراج بسته‌ی خود را عددی بررسی کنید.

**تمرین اختیاری (بدون نمره، پیش از هفته‌ی ۴ توصیه می‌شود):** تمرین‌های ۳ و ۴ lab را کامل کنید. تمرین ۳ (کمترین مربعات وزنی) پلی به بیشینه درست‌نمایی است، و تمرین ۴ پیش‌نمایشی از مسئله‌ی اصلی ارزیابی مدل است. راه‌حل‌ها در ابتدای lab هفته‌ی ۴ به‌طور مختصر بحث می‌شوند.

---

### ۱۱.۱.۱ مطالعه‌ی بیشتر

- Lewandowsky & Farrell **(۲۰۱۸)، فصل ۳.** برآورد پایه‌ی پارامتر: توابع ناهمخوانی، سطوح خطا، الگوریتم سیمپلکس و مسائل عملی مثل کمینه‌های محلی و مقادیر شروع. (منبع اصلی.)
- **قدوسیان (۱۴۰۱)**، مبانی روش‌های مستقیم در بهینه‌سازی غیرخطی: روش‌های جستجوی مستقیم برای بهینه‌سازی غیرخطی، از جمله روش‌های نوع سیمپلکس. برای دانشجویانی که نظریه‌ی پشت Nelder–Mead را می‌خواهند توصیه می‌شود [شماره‌ی فصل بررسی شود].
- Verguts **(۲۰۲۲)**: فصل بهینه‌سازی و گرادیان نزولی، مفید برای نگاه گرادیانی که در هفته‌ی ۱۵ (مدل‌های یادگیری) برمی‌گردد [شماره‌ی فصل بررسی شود].

پیش‌نمایش هفته‌ی ۴: به‌جای پرسیدن «کدام پارامترها مربع خطا را کمینه می‌کنند؟» می‌پرسیم «کدام پارامترها داده‌ی مشاهده‌شده را محتمل‌ترین می‌کنند؟» این بیشینه درست‌نمایی است. داده‌های دوجمله‌ای را درست مدیریت می‌کند و پایه‌ی AIC/BIC (هفته‌ی ۷) و استنباط بیزی (هفته‌های ۹–۱۳) است.